Mock Midterm Exam 2 (Solution Hints)

(6:35 PM – 7:50 PM : 75 Minutes)

- This exam will account for 20% of your overall grade.
- There are five (5) questions, worth 75 points in total. Please answer all of them.
- This is a *closed book*, *closed notes* exam. *No cheat sheets* are allowed.
- You are allowed to *use scratch papers* for your calculations.

GOOD LUCK!

Question	Parts	Points
1. Asymptotic Bounds	$(a), (b), (c)$	$5 + 5 + 5 = 15$
2. Recurrences.	$(a), (b), (c)$	$5 + 5 + 5 = 15$
3. Bidirectional Selection Sort	–	10
4. Partition	–	20
5. Matrix Multiply	$(a), (b)$	$10 + 5 = 15$
Total		75

Question 1. [15 Points] Asymptotic Bounds. For each of the following statements state whether it is *True* or *False*. You must justify each answer. Assume that all log's are of base 2.

(a) **[5 Points]** $n^{\frac{\log\log n}{\log n}} = \Theta\left(\log n\right)$

(b) **[5 Points]** $2^{cn} = \Omega\left(2^n\right)$, where c is a positive constant

(c) **[5 Points]** $n^{\sin n + \cos n} = \mathcal{O}\left(n^{\sqrt{2}}\right)$

Solution Hints.

(a) We have, $n^{\frac{\log\log n}{\log n}} = \left(2^{\log n}\right)^{\frac{\log\log n}{\log n}} = 2^{\log n \times \frac{\log\log n}{\log n}} = 2^{\log\log n} = \log n = \Theta\left(\log n\right)$.

Hence, the given statement is *True*.

(b) We have, $\lim_{x\to\infty}\left(\frac{2^{cn}}{2^n}\right) = \lim_{x\to\infty}\left(\frac{2^{cn}}{2^n}\right) = \lim_{x\to\infty}\left(\left(\frac{2^c}{2}\right)^n\right) = 0$ for $0 < c < 1$.

So, $2^{cn} = o\left(2^n\right)$ for $0 < c < 1$.

Hence, the given statement is *False*.

(c) We have,

$\sin n + \cos n = \sqrt{2}\left(\frac{1}{\sqrt{2}}\sin n + \frac{1}{\sqrt{2}}\cos n\right) = \sqrt{2}\left(\sin n \cos\frac{\pi}{4} + \sin\frac{\pi}{4}\cos n\right) = \sqrt{2}\sin\left(n + \frac{\pi}{4}\right) \leq \sqrt{2}$, because $\sin\frac{\pi}{4} = \cos\frac{\pi}{4} = \frac{1}{\sqrt{2}}$, $\sin x\cos y + \cos x \sin y = \sin\left(x+y\right)$ and $\sin\left(n + \frac{\pi}{4}\right) \leq 1$.

So, $n^{\sin n + \cos n} \leq n^{\sqrt{2}} = \mathcal{O}\left(n^{\sqrt{2}}\right)$.

Hence, the given statement is *True*.

Question 2. [**15 Points**] **Recurrences.** For each of the functions below either solve it using the Master Theorem or state why the Master Theorem does not apply.

(a) [**5 Points**] $T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 5T\left(\frac{n}{4}\right) + \left(2 + (-1)^n\right) n^2, & \text{otherwise.} \end{cases}$

(b) [**5 Points**] $\frac{1}{2^{\sin^2 n}} T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 2^{\cos^2 n} T\left(\frac{n}{2}\right) + n \log n, & \text{otherwise.} \end{cases}$

(c) [**5 Points**] $T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 5T\left(\frac{n^2}{3}\right) + \Theta(n), & \text{otherwise.} \end{cases}$

Solution Hints.

(a) Here, $a = 5$, $b = 4$, and $f(n) = \left(2 + (-1)^n\right) n^2$.

Since $-1 \leq (-1)^n \leq 1 \Rightarrow 1 \leq 2 + (-1)^n \leq 3 \Rightarrow n^2 \leq \left(2 + (-1)^n\right) n^2 \leq 3n^2 \Rightarrow n^2 \leq f(n) \leq 3n^2$, we have $f(n) = \Theta\left(n^2\right)$.

Then $n^{\log_b a} = n^{\log_4 5} \Rightarrow f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ for constant $\epsilon = \log_4\left(\frac{16}{5}\right) > 0$.

Also, $af\left(\frac{n}{b}\right) = 5f\left(\frac{n}{4}\right) \leq 5 \times 3\left(\frac{n}{4}\right)^2 = \left(\frac{15}{16}\right) n^2 \leq cf(n)$, where constant $c = \frac{15}{16} < 1$.

Hence, using case 3 of the Master Theorem, $T(n) = \Theta\left(f(n)\right) = \Theta\left(n^2\right)$.

(b) Multiplying both sides by $2^{\sin^2 n}$ and observing that $2^0 \leq 2^{\sin^2 n} \leq 2^1 \Rightarrow 1 \leq 2^{\sin^2 n} \leq 2 \Rightarrow 2^{\sin^2 n} = \Theta(1)$, and $2^{\sin^2 n} \times 2^{\cos^2 n} = 2^{\sin^2 n + \cos^2 n} = 2^1 = 2$, we have,

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 2T\left(\frac{n}{2}\right) + \Theta(n \log n), & \text{otherwise.} \end{cases}$$

Here, $a = 2$, $b = 2$, $f(n) = \Theta(n \log n)$, and $n^{\log_b a} = n^{\log_2 2} = n \Rightarrow f(n) = \Theta\left(n^{\log_b a} \log^1 n\right)$.

Hence, using case 2 of the Master Theorem, $T(n) = \Theta\left(n^{\log_b a} \log^2 n\right) = \Theta\left(n \log^2 n\right)$.

(c) Here, $b = \frac{3}{n}$ is not a constant and also $b = \frac{3}{n} < 1$ for $n > 3$.

Hence, the Master Theorem does not apply in this case.

Question 3. [10 Points] **Bidirectional Selection Sort.** The Bidirectional-Selection-Sort algorithm given below accepts an array $A[1..n]$ of n distinct numbers as input, and rearranges the entries of A in increasing order of value. It iterates through the array $\lfloor \frac{n}{2} \rfloor$ times, and in iteration $j \in [1, \lfloor \frac{n}{2} \rfloor]$ it finds the index of the element with the smallest value and the index of the element with the largest value in the subarray $A[j..n-j+1]$. It moves the element with the smallest value to $A[j]$ and the element with the largest value to $A[n-j+1]$ (by correctly swapping elements to make sure that no array element is destroyed in the process) before moving on to the next iteration.

Bidirectional-Selection-Sort (A)

1. ***for*** $j = 1$ ***to*** $\left\lfloor \frac{A.length}{2} \right\rfloor$ ***do***
2. // find the index of the entry with the smallest value, and the index
3. // of the entry with the largest value in $A[j..A.length - j + 1]$
4. $min = max = j$
5. ***for*** $i = j + 1$ ***to*** $A.length - j + 1$ ***do***
6. ***if*** $A[i] < A[min]$ ***then*** $min = i$
7. ***if*** $A[i] > A[max]$ ***then*** $max = i$
8. // move $A[min]$ and $A[max]$ to their correct sorted positions
9. $v_{min} = A[min]$, $v_{max} = A[max]$, $v_j = A[j]$, $v_{n-j+1} = A[n-j+1]$
10. rearrange the numbers above in the array locations above such that now $A[j] = v_{min}$ and $A[n-j+1] = v_{max}$

Analyze the worst-case running time of Bidirectional-Selection-Sort on an input of size n.

Solution Hints.

The outer ***for*** loop of lines 1–10 iterates $\lfloor \frac{n}{2} \rfloor$ times (from $j = 1$ to $j = \lfloor \frac{n}{2} \rfloor$). In each iteration of the loop lines 2, 9 and 10 together take $\Theta(1)$. In the j-th iteration of the outer loop the inner ***for*** loop of lines 5–7 iterates $(n-j+1)-(j+1)+1 = n-2j+1$ times. Lines 7–8 take $\Theta(1)$ time in each iteration of the inner loop.

Hence, the overall running time

$$\begin{aligned}
&= \sum_{j=1}^{\lfloor \frac{n}{2} \rfloor} \left(\Theta(1) + \sum_{i=j+1}^{n-j+1} \Theta(1) \right) \\
&= \sum_{j=1}^{\lfloor \frac{n}{2} \rfloor} \left(\Theta(1) + \Theta(n - j + 1 - (j+1) + 1) \right) \\
&= \sum_{j=1}^{\lfloor \frac{n}{2} \rfloor} \left(\Theta(1) + \Theta(n - 2j + 1) \right) \\
&= \sum_{j=1}^{\lfloor \frac{n}{2} \rfloor} \left(\Theta(n - 2j + 1) \right) = \Theta\left(\sum_{j=1}^{\lfloor \frac{n}{2} \rfloor} (n - 2j + 1) \right) \\
&= \Theta\left(\sum_{j=1}^{\lfloor \frac{n}{2} \rfloor} (n+1) - 2\sum_{j=1}^{\lfloor \frac{n}{2} \rfloor} j \right) \\
&= \Theta\left((n+1)\left\lfloor \frac{n}{2} \right\rfloor - 2 \times \frac{1}{2} \times \left(\left\lfloor \frac{n}{2} \right\rfloor + 1 \right) \left\lfloor \frac{n}{2} \right\rfloor \right) = \Theta\left(\left(n - \left\lfloor \frac{n}{2} \right\rfloor - 1\right) \left\lfloor \frac{n}{2} \right\rfloor \right) \\
&= \Theta\left(\left(\left\lceil \frac{n}{2} \right\rceil - 1 \right) \left\lfloor \frac{n}{2} \right\rfloor \right) \\
&= \Theta\left(n^2\right)
\end{aligned}$$

QUESTION 4. [20 Points] Partition. Given an input array $A[p..r]$ of $n = r - p + 1$ numbers, the *Partition* algorithm we saw in the class rearranges the items of A in-place such that for some $q \in [p, r]$ everything in $A[p..q-1]$ is $\leq A[q]$ and everything in $A[q+1..r]$ is $\geq A[q]$.

Recall that the outermost ***for*** loop of the algorithm iterates from $j = p$ to $j = r - 1$.

Suppose the following array $A[1..10]$ is given as input to the algorithm. Please show the state of this array after each iteration of the outermost ***for*** loop. Please also show the final state of $A[1..10]$ before the algorithm terminates.

	1	2	3	4	5	6	7	8	9	10
A	14	8	10	5	19	15	2	12	9	11

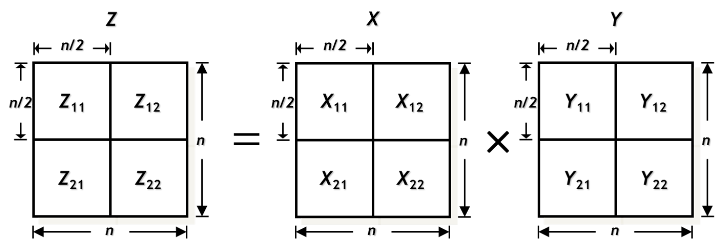
There must be one line for each value of j. Please start each line with the j value followed by a ":" and then write down the items (separated by commas, no fancy boxes are needed) of the input array in the order they will appear in $A[1..10]$ at the end of the j-th iteration of the loop. The last line should start with the prefix "final state:" and show the final state of the array.

SOLUTION HINTS.

$$
\begin{array}{rrrrrrrrrrr}
j = 1: & 14, & 8, & 10, & 5, & 19, & 15, & 2, & 12, & 9, & 11 \\
j = 2: & 8, & 14, & 10, & 5, & 19, & 15, & 2, & 12, & 9, & 11 \\
j = 3: & 8, & 10, & 14, & 5, & 19, & 15, & 2, & 12, & 9, & 11 \\
j = 4: & 8, & 10, & 5, & 14, & 19, & 15, & 2, & 12, & 9, & 11 \\
j = 5: & 8, & 10, & 5, & 14, & 19, & 15, & 2, & 12, & 9, & 11 \\
j = 6: & 8, & 10, & 5, & 14, & 19, & 15, & 2, & 12, & 9, & 11 \\
j = 7: & 8, & 10, & 5, & 2, & 19, & 15, & 14, & 12, & 9, & 11 \\
j = 8: & 8, & 10, & 5, & 2, & 19, & 15, & 14, & 12, & 9, & 11 \\
j = 9: & 8, & 10, & 5, & 2, & 9, & 15, & 14, & 12, & 19, & 11 \\
\text{final state}: & 8, & 10, & 5, & 2, & 9, & 11, & 14, & 12, & 19, & 15
\end{array}
$$

Question 5. [**15 Points**] **Matrix Multiply.** The *product* of two $n \times n$ matrices $X[1..n, 1..n]$ and $Y[1..n, 1..n]$ is another $n \times n$ matrix $Z[1..n, 1..n]$ with $Z[i, j] = \sum_{k=1}^{n} X[i, k] \cdot Y[k, j]$ for $1 \le i, j \le n$.

The Rec-Matrix-Multiply algorithm given below computes the product of two square matrices using recursive divide-and-conquer. The algorithm accepts two $n \times n$ square matrices $X[1..n, 1..n]$ and $Y[1..n, 1..n]$ as inputs, and recursively generates the product of X and Y in $Z[1..n, 1..n]$. Assume for implicity that $n = 2^r$ for some integer $r \ge 0$. If $n > 1$, X_{11}, X_{12}, X_{21} and X_{22} denote the top-left, top-right, bottom-left and bottom-right quadrants of X, respectively. Similarly for Y and Z. Each quadrant is an $\frac{n}{2} \times \frac{n}{2}$ submatrix of the parent $n \times n$ matrix.




```
Rec-Matrix-Multiply ( X, Y, Z, n )
if n = 1 then
    Z = X · Y                                        // a 1 × 1 matrix is just a single number
else
    allocate two n × n matrices U and V
    // CONQUER:
    Rec-Matrix-Multiply ( X11, Y11, U11, n / 2 )     // product of X11 and Y11 is placed in U11
    Rec-Matrix-Multiply ( X11, Y12, U12, n / 2 )     // product of X11 and Y12 is placed in U12
    Rec-Matrix-Multiply ( X21, Y11, U21, n / 2 )     // product of X21 and Y11 is placed in U21
    Rec-Matrix-Multiply ( X21, Y12, U22, n / 2 )     // product of X21 and Y12 is placed in U22
    Rec-Matrix-Multiply ( X12, Y21, V11, n / 2 )     // product of X12 and Y21 is placed in V11
    Rec-Matrix-Multiply ( X12, Y22, V12, n / 2 )     // product of X12 and Y22 is placed in V12
    Rec-Matrix-Multiply ( X22, Y21, V21, n / 2 )     // product of X22 and Y21 is placed in V21
    Rec-Matrix-Multiply ( X22, Y22, V22, n / 2 )     // product of X22 and Y22 is placed in V22
    // COMBINE: matrix Z is the sum of U and V
    for i = 1 to n do
      for j = 1 to n do
        Z[i, j] = U[i, j] + V[i, j]
```

(*a*) [**10 Points**] Write a recurrence relation describing the running time of Rec-Matrix-Multiply when multiplying two $n \times n$ matrices.

(*b*) [**5 Points**] Solve the recurrence relation from part (*a*).

SOLUTION HINTS.

(*a*) When $n = 1$, line 2 is executed which takes $\Theta(1)$ time. Otherwise, lines 4–17 are executed. Line 4 takes $\mathcal{O}\left(n^2\right)$ time and lines 15-17 take $\Theta\left(n^2\right)$ time, and thus taking $\Theta\left(n^2\right)$ time in total. The algorithm calls itself recursively 8 times in lines 6–13 on input matrices of size $\frac{n}{2} \times \frac{n}{2}$.

Let $T(n)$ be the running time of the algorithm on an input matrices of size $n \times n$. Then

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1; \\ 8T\left(\frac{n}{2}\right) + \Theta\left(n^2\right), & \text{otherwise.} \end{cases}$$

(*b*) Here, $a = 8$, $b = 2$, $f(n) = \Theta\left(n^2\right)$, and $n^{\log_b a} = n^{\log_2 8} = n^3 \Rightarrow f(n) = \Theta\left(n^{\log_b a - \epsilon}\right)$, where constant $\epsilon = \log_2 2 = 1 > 0$.

Hence, using case 1 of the Master Theorem, $T(n) = \Theta\left(n^{\log_b a}\right) = \Theta\left(n^3\right)$.